

XIPHOS: Adaptive In-Vehicle Intrusion Detection via Unsupervised Graph Contrastive Learning

Qiguang Jiang^{ID}, Kai Wang^{ID}, *Member, IEEE*, Yuliang Wei^{ID}, Hongri Liu^{ID}, and Bailing Wang^{ID}, *Member, IEEE*

Abstract—As vehicles have become increasingly connected and intelligent, attacks against in-vehicle networks (IVNs) are becoming more prevalent and pose a great threat to vehicle security and occupant safety. Intrusion detection techniques utilizing deep learning models have become a common approach to secure IVNs. However, existing work has shown some weaknesses. 1) They are unable to directly extract the rich information hidden in the data behavioral patterns. 2) The effectiveness of most supervised models depends on balanced data distributions and high-quality labels, whereas the current state of real-world datasets does not match these demands. 3) The performance of unsupervised learning models is inferior to supervised methods, accompanied by unstable or unpredictable results. In this paper, we design and implement XIPHOS, a novel and adaptive IVN intrusion detection mechanism that is capable of achieving efficient detection performance in the unsupervised environment. XIPHOS utilizes the principle of mutual information maximization to extract as many potential data invariants as possible. By detecting abnormal system behaviors through error offsets of clustered combinations of feature units, XIPHOS is able to perform both graph-level representation and node-level representation from IVN data. In addition, the adaptiveness of XIPHOS is indicated by its ability to update the model parameters over time at different detection scenarios. Experimental results on widely used datasets show that XIPHOS has greater advantages over existing methods in terms of both detection performance and freedom from attack labeling data dependencies. The code is available at <https://github.com/wangkai-tech23/XIPHOS>

Index Terms—Controller area network, unsupervised intrusion detection, graph contrastive learning, graph-theory, in-vehicle network.

I. INTRODUCTION

THE popular intelligent connected vehicles (ICVs) have already been equipped with in-vehicle communication devices and a variety of embedded computing devices containing hundreds of sensors and actuators managed by electronic control units (ECUs), providing users with a variety of intelligent and comfort services [1], [2].

Received 28 August 2024; revised 8 May 2025 and 18 August 2025; accepted 25 September 2025. Date of current version 8 October 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62272129 and in part by Taishan Scholar Foundation of Shandong Province under Grant tsqn202408112. The associate editor coordinating the review of this article and approving it for publication was Prof. Kemal Akkaya. (*Corresponding author: Kai Wang.*)

Qiguang Jiang and Kai Wang are with the School of Computer Science and Technology, Harbin Institute of Technology, Weihai 264209, China, and also with Shandong Key Laboratory of Industrial Network Security, Weihai 264209, China (e-mail: jiangqiguang_971@163.com; dr.wangkai@hit.edu.cn).

Yuliang Wei, Hongri Liu, and Bailing Wang are with the Harbin Institute of Technology, Weihai 264209, China, also with Qingdao Research Institute, Qingdao 266109, China, and also with Shandong Key Laboratory of Industrial Network Security, Weihai 264209, China (e-mail: wei.yl@hit.edu.cn; liuhr@hit.edu.cn; wbl@hit.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3616624

The central communication channel between the ECUs is the Controller Area Network (CAN), which is the best known and most widely used protocol in the automotive industry [3], [4]. The CAN bus has real-time communication with minimal time delay and priority arbitration mechanism and completes the data exchange between the ECUs by providing a public network with standardized protocols.

However, there are well-known security flaws in the protocol design that make it possible for attacks on the IVN with the ability to seriously jeopardize driving safety [5], [6], [7]. Notably, recent studies have shown that adversaries can inject forged CAN messages to manipulate powertrain control, disable safety systems, or affect driver assistance functions, even in modern production vehicles. These vulnerabilities have driven a surge of research in recent years toward intrusion detection and anomaly analysis techniques for securing CAN bus [8], [9].

Thus, in order to protect IVNs from injection attacks, existing approaches can be summarized into message authentication and intrusion detection. Since the CAN bus is almost ubiquitous as the backbone of the IVN in modern vehicles, changing the CAN protocol (such as adding message authentication, data encryption [10]) would require modifying the integration approach of all vehicle manufacturers, which is clearly impractical.

Fortunately, as another promising technology, intrusion detection can provide security without modifications to the CAN protocol, and can be categorized into specification-based methods and machine learning (ML)-based methods [11]. The former constructs a knowledge base by capturing and creating all attack signatures in advance and detects attacks by monitoring the current network traffic based on the signatures in the predefined knowledge base [12]. However, it is unable to detect new and previously unseen attacks such as zero-day threats, and detection accuracy can only be ensured by continuously maintaining and updating the knowledge base.

For instance, as a representative of specification-based methods, knowledge graph-based methods [13], [14] often rely on predefined knowledge bases and known attack signatures, limiting their adaptability to emerging or evolving threats. Moreover, such methods incur high construction and maintenance costs and exhibit inherent limitations in modeling temporal dependencies and achieving cross-platform generalization.

In contrast, ML-based approaches, including traditional machine learning, regression algorithm, deep learning and reinforcement learning [15], [16], are becoming increasingly attractive for building intrusion detection systems. Among

them, as the state-of-the-art method, deep learning-based methods have become the mainstream of IVN intrusion detection due to their unique ability to discover hidden patterns in massive data, which helps to improve classification performance. Based on a fair comparative performance analysis in [16], we summarize the following limitations of deep learning-based IVN intrusion detection methods.

(1) Most of the existing intrusion detection methods do not change the original multidimensional time-series data format in preprocessing and detection, which cannot further reflect the regularity between CAN message's identifiers (CAN IDs) and hidden patterns in data flow, resulting in poor detection performance. (2) In practice, labeling CAN bus datasets is often costly, and attack samples are typically much fewer than normal ones. This class imbalance significantly hinders the effectiveness of supervised learning and limits its potential for further performance improvement. (3) It is possible for the unsupervised model to overfit the data or be affected by low-quality or noisy data, which can degrade the performance of the model, leading to unstable results and difficulty in reproducing results. (4) Existing intrusion detection methods lack architectural designs that support differentiated adaptation to vehicle-cloud resources. Deep learning-based intrusion detection models, in particular, need to demonstrate stable operation in resource-constrained, real-time in-vehicle environments to validate their practical feasibility.

In this paper, we address the above four problems by proposing XIPHOS, a novel unsupervised and adaptive IVN intrusion detection mechanism that utilizes graph contrastive representation learning and adaptive classification methods to identify injection attacks from a large number of CAN messages. Firstly, XIPHOS starts by converting raw CAN data into graph-structured data based on the logical topology characteristics and data flow coupling characteristics between vehicle communication nodes. Then, XIPHOS quantifies potential similarities between graph-structured data and constructs graph or node vector embeddings in the graph contrastive learning module in a way that maximizes mutual information. This module does not require any labeled data and additionally considers the fusion of local and global information. Finally, an adaptive pre-training is performed based on a moderate amount of normal data samples and feature mapping with autoencoder-based error reconstruction in response to data fluctuations. A Single Gaussian Model (SGM) is used to analyze and obtain the final classification results.

XIPHOS is designed to be adaptive and scalable. Depending on the application scenarios, XIPHOS is free to select some normal samples in the adaptive hierarchical clustering classification module, i.e., it can choose to restart training at any time to adapt to the data changes over time. Next, depending on the downstream task requirements, XIPHOS can perform different granularity of detection, i.e., realizing graph classification or node classification tasks. While XIPHOS is designed to perform feature extraction without labeling in the graph contrastive learning module, it can alternatively be implemented for semi-supervised and supervised learning.

We implement XIPHOS and evaluate the performance on two different CAN datasets with injection attack: the Car Hacking dataset [17] and the ROAD dataset [18]. The ROAD dataset contains attacks on real vehicles, while the Car Hacking dataset is simulated in the in-vehicle environment. The evaluation results show that XIPHOS achieves better performance on IVN intrusion detection with a much lower dependence on labels than the state-of-the-art methods.

In summary, our main contributions are as follows.

1) We propose XIPHOS, an unsupervised and adaptive detection approach based on mutual information maximization, which is the first adaptive intrusion detection framework using unsupervised graph contrastive learning in IVNs, and does not require attack labels throughout the process.

2) We introduce an enhanced mutual information estimation strategy by extending graph representations from the original node-graph to node-node plus graph-graph, thereby improving feature expressiveness and discrimination, especially for capturing complex temporal dependencies in CAN traffic.

3) We conduct extensive empirical validation on multiple real-world CAN datasets under a vehicle-cloud collaborative architecture, evaluating both supervised and unsupervised baselines in terms of detection accuracy and inference latency on automotive-grade hardware. The results demonstrate that XIPHOS not only achieves high detection performance but also satisfies real-time operational requirements, effectively bridging the gap between academic research and practical in-vehicle deployment.

The remainder of this paper is organized as follows: Section II introduces the background knowledge and provides the necessary preliminaries. Section III describes our proposed XIPHOS. We show the experiments and results in section IV, followed by the discussion in section V. Related work is given in section VI. We conclude the paper in section VII.

II. BACKGROUND AND PRELIMINARIES

In this section, we introduce the CAN protocol and threats in § II-A and § II-B. Then, we provide the definitions and preliminaries needed in § II-C and § II-D.

A. CAN Protocol

According to the ISO 11898 standard, the CAN protocol covers the physical layer and the transport layer. Various classic in-vehicle networks such as CAN, CAN with Flexible Data Rate (CAN FD), Local Interconnect Network (LIN), FlexRay, and Media Oriented System Transport (MOST) are connected through gateways. As the de facto communication standard for ECUs in IVN systems, CAN is a broadcast-like serial data communication protocol to support message passing between ECUs [19]. It uses differential voltage signals to denote Dominant bit (0) and Recessive bit (1). Each ECU executes vehicle control logic on its microcontroller and handles CAN message reception (bitstream interpretation and ID matching) and transmission (encoding messages into differential voltage signals via the CAN controller).

CAN messages have four main types of frames: data frames, remote frames, overload frames, and error frames. The data

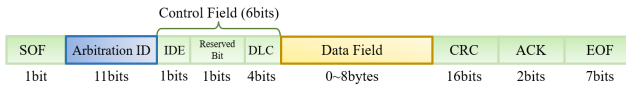


Fig. 1. CAN frame.

frame is used to send data, consisting of the following: start of frame (SOF), arbitration field, control field, data field, cyclic redundancy check (CRC) field, acknowledgment (ACK) field and end of frame (EOF). The arbitration field determines the priority of the message, with 11-bit identifier in standard frames and 29-bit identifier in extended frames. The control field specifies whether the frame is in standard or extended format and indicates the length of the data payload. The data field carries the actual transmitted content and is encoded in Big-Endian format. The CRC field is responsible for detecting bit errors that may occur due to noise or transmission conflicts. The ACK field is used to confirm whether the message was received correctly, and the EOF marks the end of the frame [4], [20]. **Arbitration ID** and **Data Field** are relevant to the scope of this paper. The complete figure of the CAN message data frame is shown in Fig. 1. It is important to note that data transmitted over the CAN bus is inherently untagged. In existing CAN datasets, labels are manually assigned post hoc, with each CAN frame annotated as either normal or abnormal.

When multiple ECUs attempt simultaneous transmission, the CAN protocol employs an arbitration mechanism to resolve contention, granting bus access to the message with the highest priority, indicated by the lowest Arbitration ID. Lower-priority frames withdraw and enter receiver mode, allowing only the winning node to transmit. After a 3-bit inter-frame space (IFS), nodes that have buffered data or previously lost arbitration will initiate another round of access arbitration. The CAN ID not only determines transmission priority but also identifies message content. Typically, each ECU is assigned a fixed set of IDs, which are transmitted at regular intervals to update signal values. The data field (payload) carries up to 8 bytes of message content. The CAN protocol incorporates robust error-handling mechanisms essential for fault tolerance [4]. Errors such as frame loss, retransmission, and flag triggering are detectable, enabling nodes to respond with appropriate corrective actions [20].

B. Threats in CAN Bus

An attacker can use physical ports (e.g., On-Board Diagnostic (OBD) ports, USB ports, etc.) or wireless networks (short-range wireless interfaces represented by Bluetooth, remote wireless networks such as broadcasting channels, addressable channels, etc.) which adversaries may use to intrude into the control of the CAN bus or ECU.

1) *Compromised ECU*: In order to understand the complexity of an attack more clearly and quickly, we use the attack model defined in [18] along with widely used terminology.

Weakly compromised ECU can be suspended by the adversaries for any message transmission.

Fully compromised ECU is under full control of the attacker, including sending fake messages and accessing the node's memory.

2) *Injection Attack on CAN*: Research [18] divides the attacks into three general categories, including fabrication attacks, suspension attacks, and masquerade attacks.

Fabrication Attacks are injected from the fully compromised ECUs using forged IDs and Data Fields. DoS attack, fuzzy attack, and targeted ID attack are three typical fabrication attacks. In these attacks, the adversary can inject arbitrary messages into the bus while the legitimate ECU is still sending legitimate messages, triggering the message conflict problem [21]. Simple fabrication attacks are therefore bound to bring about periodic changes in CAN messages. It is especially easy to detect attacks that send messages at a very high frequency.

Suspension Attacks only need to prevent a weakly compromised ECU from transmitting messages. They can cause damage not only to the weakly compromised ECU itself, but also to other ECUs that rely on its information to function.

Masquerade Attacks are the most advanced and sophisticated types of attacks, requiring substantial hacking expertise, access to system-level resources, and comprehensive knowledge of the target vehicle's internal architecture. In this attack, the adversary first disables the target ECU, rendering it functionally inactive, i.e., suspending its message transmissions and placing it in a weakly compromised state. Simultaneously, a fully compromised ECU is deployed to impersonate the unavailable target, injecting spoofed messages with the same ID at a realistic transmission frequency. These malicious messages can deceive other ECUs into executing unintended actions, potentially resulting in system malfunctions. A key characteristic of the masquerade strategy is that it enables the injection of malicious messages with targeted IDs without causing message conflicts (e.g., without changing temporal patterns of the message transmission), thereby preserving the temporal integrity of the communication flow. Notably, masquerade attacks typically operate on specific signal fields within the CAN frame rather than replacing entire data payloads, making them significantly more precise and stealthy, and thus more difficult to detect by conventional intrusion detection mechanisms.

C. Definition

1) *CAN Messages Definition*: A CAN message refers to a structured data frame transmitted over the CAN bus. Each message typically contains an Arbitration ID, a payload of up to 8 bytes, and optional metadata such as timestamps or labels for intrusion detection purposes.

Definition 1: For modeling purposes, a CAN data stream can be represented as a multivariate time series $\Xi = \{\xi_1, \xi_2, \dots, \xi_t, \dots, \xi_T\} \in \mathbb{R}^{T \times 10}$, where each message at time t is denoted as:

$$\xi_t = \{ID, d_1, d_2, \dots, d_8, d_l\} \in \mathbb{R}^{10}, \quad t \in \{1, 2, \dots, T\} \quad (1)$$

Therein, ID is the arbitration ID of the CAN message, d_1 to d_8 are the byte-level payload values in the Data Field, and d_l is a label value indicating the type of attack (or normal message). For notational simplicity, we rewrite ξ_t as $\xi_t = \{d_0, d_1, d_2, \dots, d_8, d_l\}$. In this work, d_0 and the arbitration ID are used interchangeably. We next formalize a key term used in our proposed algorithm.

Definition 2: We consider W CAN messages arranged according to the normal communication as a window. W is the window size [22]. Assume that there are L windows in total. w_l denotes the matrix consisting of CAN messages within the l -th window after preprocessing.

$$w_l = \{\xi_{l*W}, \xi_{l*W+1}, \dots, \xi_{l*W+(W-1)}\} \quad (2)$$

where $l \in \{0, 1, 2, \dots, L-1\}$.

2) *Problem Formulation:* This paper is dedicated to formulating the IVN intrusion detection task as a graph learning modeling problem, and thus the problem can be defined as follows.

Definition 3 (Unsupervised Graph Representation Learning): Given a set of graphs $\mathcal{G} = \{G_1, G_2, \dots, G_{|\mathcal{G}|}\}$, we expect to learn a d -dimensional representation of every single graph $G_i \in \mathcal{G}$ ($d \in \mathbb{Z}^+$). Existing graph embedding tasks are categorized into node-level embedding and graph-level embedding, which typically learn by aggregating features of their neighboring nodes. Thus, we denote the embedding of graph G_i as $H(G_i) \in \mathbb{R}^d$, and node embedding or patch representations of graph G_i as $h(G_i) \in \mathbb{R}^{|G_i| \times d}$, where $|G|$ indicates the number of nodes.

D. Preliminaries

1) *Mutual Information (MI):* As distinct from correlation, MI captures non-linear statistical dependencies between variables, and thus can act as a measure of true dependence [23]. In brief, MI quantifies the dependence of two random variables X and Y in the form of

$$I(X; Y) = D_{KL}(\mathbb{P}_{XY} \| \mathbb{P}_X \otimes \mathbb{P}_Y)$$

where $\mathbb{P}_{XY}$ is the joint probability distribution, and $\mathbb{P}_X$ and $\mathbb{P}_Y$ are marginal distribution. D_{KL} is the KL-divergence with the following expression:

$$D_{KL}(\mathbb{P} \| \mathbb{Q}) := \int_x p(x) \log \left(\frac{p(x)}{q(x)} \right) dx = \mathbb{E}_{\mathbb{P}} \left[\log \frac{d\mathbb{P}}{d\mathbb{Q}} \right]$$

2) *Mutual Information Maximization:* Direct maximization of $I(X; Y)$ via KL-based variational bounds, such as the Donsker–Varadhan representation, is theoretically well-founded but often unstable in practice, particularly in high-dimensional settings, due to the unbounded nature of the KL divergence and the high variance of its estimators. To obtain a stable and bounded training objective, we adopt the variational f -divergence framework [24] and employ the Jensen–Shannon (JS) divergence as the optimization criterion. Specifically, we estimate $D_{JS}(\mathbb{P}_{XY} \| \mathbb{P}_X \otimes \mathbb{P}_Y)$ using a discriminator-based variational lower bound, providing a numerically stable surrogate for dependence estimation between X and Y . This choice does not alter the definition of MI; rather, it replaces the KL-based estimator with a JS-based objective that yields bounded gradients, better conditioning, and improved empirical robustness.

The variational formulation of the JS divergence allows it to be expressed via a discriminator scoring function s , as shown in [24]. This directly leads to Theorem 1, where we present a variational lower bound for MI based on the JS divergence and show that the discriminator objective yields a tractable and optimizable bound on D_{JS} .

TABLE I
THE DISTRIBUTION OF THE CAR HACKING DATASET

Data Type	Normal Data Quantity	Attack Data Quantity	Attack Data Proportion	Total Data
Normal	988,987	0	0%	988,987
DoS Attack	3,078,250	587,521	16%	3,665,771
Fuzzy Attack	3,347,013	491,847	13%	3,838,860
GEAR Attack	3,845,890	597,252	13%	4,443,142
RPM Attack	3,966,805	654,897	14%	4,621,702

Theorem 1 ((JS MI Lower Bound): Let $\mathbb{P}$ and $\mathbb{Q}$ be two distributions. Define a discriminator scoring function s with output $D_s(x) = \sigma(s(x))$, where $\sigma(\cdot)$ is the sigmoid function and x denotes a pair of input features.

From Eq. (8) in [24], the variational formulation of the JS divergence defines the discriminator optimization objective as

$$\mathcal{J}(s) = \mathbb{E}_{z \sim \mathbb{P}} [\log D_s(z)] + \mathbb{E}_{z \sim \mathbb{Q}} [\log (1 - D_s(z))]$$

Using the variational representation of $D_{GAN}(\mathbb{P} \| \mathbb{Q})$ in Table I of [24], we obtain

$$D_{JS}(\mathbb{P} \| \mathbb{Q}) \geq \frac{1}{2} (\mathcal{J}(s) + \log 4), \quad \forall s$$

indicating that for any non-optimal s , the above expression provides a variational lower bound on the JS divergence that can be optimized directly.

III. XIPHOS FRAMEWORK

XIPHOS is an unsupervised and adaptive intrusion detection for IVNs, which leverages graph contrastive learning that maximizes local and global MI and adaptive hierarchical clustering mapping to achieve high-performance intrusion detection in the absence of attack label information, providing meaningful inspiration for unsupervised graph learning. XIPHOS consists of three major components: (a) graph-structured data generation (§ III-A), (b) graph contrastive representation learning (§ III-B) and (c) adaptive hierarchical clustering classification (§ III-C). In the training process, XIPHOS transforms raw data from (a), learns graph embeddings with (b), and learns benign behavioral patterns in (c). In the inference process, XIPHOS transforms the target data with (a), obtains the graph embeddings in (b), and detects anomalies with (c). The overview of XIPHOS is depicted in Fig. 2.

Although CAN messages are typically recorded as multidimensional time-series data, this format fails to capture their inherent structural regularities. To address this, component (a) transforms raw CAN bus data into spatio-temporally correlated graph-structured data, comprising feature and adjacency matrices that reflect logical topology and coupling patterns among communication nodes. Further details are provided in [25].

The graph-structured data is subsequently processed by the graph contrastive learning module (b) to obtain graph-level or node-level embeddings. The encoder quantifies potential similarities among graph instances by maximizing MI, while integrating both local and global node information to enhance representation quality.

Adaptive hierarchical clustering classification module (c) is trained exclusively on benign features generated by

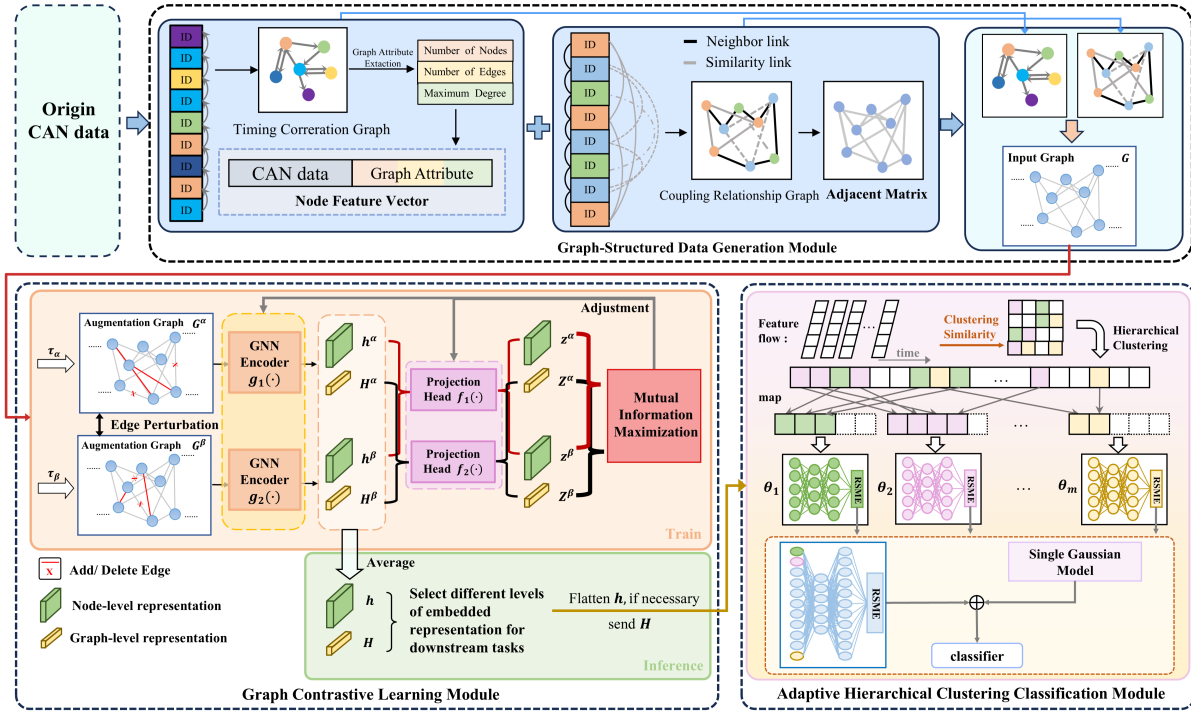


Fig. 2. The architecture of XIPHOS.

the contrast learning module to model normal message behavior. Feature combinations are processed by multiple independent self-encoders to compute root mean square error (RMSE) values, which are then evaluated using a SGM to determine thresholds and identify anomalies. This module further supports adaptive detection by continuously updating its feature mapping strategy in response to varying data distributions.

The following sections provide a detailed explanation of the operation of these three modules and illustrate how their integration enables unsupervised and adaptive intrusion detection without reliance on labeled attack data.

A. Graph-Structured Data Generation Module

As a complex non-Euclidean data structure, a graph is better suited than traditional two-dimensional Euclidean representations for modeling intricate relationships. In particular, graph-theoretic approaches are effective in capturing interaction patterns among ECUs. Prior work [22] proposed a graph construction method based on CAN bus data distribution, leveraging periodicity-induced regularities of CAN messages. This approach, known for its interpretability and simplicity, serves as an inspiration for our design. To more accurately represent real-world graph data, it is often necessary to construct multiple graph views that capture different types of relationships among nodes. Accordingly, we introduce two graph views in this study: one to represent message timing dependencies and another to capture ID association relationships, jointly used to generate graph-structured data $G_l = (X_l, A_l)$, $l = 0, 1, \dots, L - 1$.

We construct **Timing Correlation Graphs** (TCGs) to capture the long-term distributional patterns of CAN messages

and derive node feature vectors from their structural attributes. TCGs characterize temporal interaction features within CAN message streams, providing insights into variations in message transmission probabilities. In parallel, **Coupling Relationship Graphs** (CRGs) are constructed to generate adjacency matrices, where edges encode direct interactions between CAN messages based on identifier-level associations. While both TCGs and CRGs serve as novel graph-based views of CAN traffic, they complement each other by representing distinct aspects of ECU communication: TCGs emphasize temporal distribution patterns, whereas CRGs highlight coupling relationships among message flows.

1) **TCG:** According to definition 2, the multivariate CAN time series is segmented into L temporal windows. Within each window, each distinct CAN ID is represented as a node, and directed edges are established to capture transitions between IDs, permitting bidirectional connections. As shown in Fig. 2, the latter ID value points to the previous ID value, all messages in the l -th window generate a directed graph $G_l^{(T)}$, with the number of nodes equal to the number of different IDs. This yields a set of TCGs $G_1^{(T)}, \dots, G_L^{(T)}$ for L windows.

The statistical properties of TCG (such as the average weight, the max degree, the number of nodes, the number of edges, the max weight) capture the global structure within the current window, which can be merged with the local payload of each CAN message.

Specifically, we extract the node number, edge number, maximum degree from single constructed TCG $G_l^{(T)}$ of l -th window as $x_l^{(n)}, x_l^{(e)}, x_l^{(d)}$, which supplement every local CAN message ξ_l as node feature vector x_l . We obtain $x_l = \{d_0, d_1, d_2, \dots, d_8, x_l^{(n)}, x_l^{(e)}, x_l^{(d)}, d_l\} \in \mathbb{R}^{13}$, x_l in l -th window. Hence, node feature matrix $X_l = \{x_{l*W}, \dots, x_{l*W+(W-1)}\}$.

2) *CRG*: To capture short-term associations in CAN traffic, it requires us to enhance the correlation between the same IDs in the CAN delivery mechanism. As illustrated in Fig. 1, such coupling is reflected in the contextual relevance between adjacent messages and the similarity among messages sharing the same ID. Thus, we construct the CRGs $G_1^{(C)}, \dots, G_L^{(C)}$ for L windows as follows. The window segmentation is aligned with that defined for TCGs in Definition 2.

There are N nodes in each $G_l^{(C)}$, because we treat each CAN message as a node. The edges in CRG $G_l^{(C)}$ have two kinds of connections. The first is the connection between two consecutive CAN messages with generally different CAN ID values. The other is an edge that connects two nodes with the same CAN ID values.

In the CRG generation phase, as illustrated in Fig. 2, the gray solid, dashed, and dot-dash links to the right of each CAN ID represent the similarity relationships among messages with the same ID, while the black links to the left indicate the temporal adjacency between consecutive CAN messages. Hence, we get $A_l = G_l^{(C)}$ and the final graph-structure data is $G_l = (X_l, A_l)$, $l = 0, 1, \dots, L - 1$.

In summary, we propose timing correlation statistical mechanism (TCG) and a relationship-aware message passing paradigm (CRG) to reflect the long-term statistical distribution and coupling relationship, generating graph-structured data $\mathcal{G} = \{G = (X, A)\}$, $|\mathcal{G}| = L$.

B. Graph Contrastive Learning Module

The central focus of the graph comparison learning module is to train a powerful graph neural network (GNN) encoder to characterize graph nodes. Building upon the recent augmented graph contrastive learning method [26], we optimize the MI metric in a way that strengthens the MI of local nodes in order to utilize the graph information more comprehensively.

Formally, the k -th layer of a GNN on graph G can be represented as

$$h_v^{(k)} = \text{COMBINE}^{(k)}(h_v^{(k-1)}, a_n^{(k)}) \quad (3)$$

$$a_n^{(k)} = \text{AGGREGATION}^{(k)}(h_u^{(k-1)}, u \in \mathcal{N}(v)) \quad (4)$$

where $h_v^{(k)}$, $v \in G$ is the patch feature embedding of node v through k layers, $\mathcal{N}(v)$ is the neighborhood of node v . Besides, $\text{AGGREGATION}^{(k)}(\cdot)$ and $\text{COMBINE}^{(k)}(\cdot)$ are component functions of the GNN layer.

Define parameters of GNN as ϕ . Hence, the patch representation at the k -th layer on all nodes in graph G is denoted as

$$h_\phi^{(k)}(G) = \text{CONCAT}(h_v^{(k)}), v \in G \quad (5)$$

Therein, $\text{CONCAT}(\cdot)$ denotes the concatenation function that aggregates feature vectors across k GNN layers into a single representation, capturing multi-hop neighborhood information centered at each node [27].

After K layers propagation, the final output embedding for the whole graph G is

$$H_\phi(G) = \text{READOUT}\left(h_\phi^{(k)}(G)\right)_{k=1}^K \quad (6)$$

The function $\text{READOUT}(\cdot)$ can be implemented as a simple permutation-invariant operation such as average pooling, or as a more sophisticated graph-level aggregation function designed to capture complex structural information. For simplicity, we rewrite $h_\phi^{(K)}(G)$ as $h_\phi(G)$.

In contrast to previous GNN-based learning, graph contrastive learning reinforces and maximizes non-linear statistical dependencies between two augmented views by comparing the representation of one view against that of the other. In our method, we adopt edge perturbation as the augmentation strategy, where each edge is independently added or removed according to an i.i.d. uniform distribution, thereby generating two augmented graphs G', G'' from the original graph G . This operation perturbs the connectivity structure of G by randomly modifying a certain proportion of edges. Edge perturbation serves as an augmentation/noise addition to make graph contrastive learning model less fragile, which demonstrates robustness to edge connectivity pattern variations.

After obtaining node-level representations $h_\phi(G'), h_\phi(G'')$, and graph-level representations $H_\phi(G'), H_\phi(G'')$, we apply a nonlinear transformation $f(\cdot)$, referred to as the projection head, to map these augmented representations into a separate latent space. Specifically, we design $f_i(\cdot)$ and $f_g(\cdot)$ as two-layer multilayer perceptrons (MLPs) with PReLU activations. Define the parameters of $f(\cdot)$ as ϑ collectively. The resulting projected embeddings are denoted as $z = f_i(h_\phi(G))$ for the node-level representation, and $Z = f_g(H_\phi(G))$ for the graph-level representation.

1) *Training Process*: The objective of graph contrastive learning is to train encoders and capture informative graph representations suitable for downstream tasks. To this end, our approach is inspired by the Deep InfoMax framework [28], and seeks to maximize MI between node–node and graph–graph embeddings across two augmented views. This dual-level contrast yields more fine-grained local representations, and outperforms methods that rely solely on graph–graph or node–graph contrast.

According to [24], we give the Jensen-Shannon MI estimator on local and global pairs, maximizing the estimated MI as follows:

$$(\hat{\phi}, \hat{\vartheta}, \hat{\psi})_{\mathcal{G}} = \arg \max_{\phi, \vartheta, \psi} \sum_{G \in \mathcal{G}} \left(I_{\phi, \vartheta, \psi}(z', z'') + I_{\phi, \vartheta, \psi}(Z', Z'') \right) \quad (7)$$

where $I_{\phi, \vartheta, \psi}$ is the MI estimator modeled by discriminator T_ψ , and T_ψ is a discriminator function modeled by a neural network with parameters ψ . Moreover, $z' = f_i(h_\phi(G'))$, $z'' = f_i(h_\phi(G''))$, $Z' = f_g(H_\phi(G'))$, $Z'' = f_g(H_\phi(G''))$.

We have

$$\begin{aligned} & I_{\phi, \vartheta, \psi}(f_i(h_\phi(G')), f_i(h_\phi(G''))) \triangleq \\ & \mathbb{E}_{\mathcal{P}} \left[-sp \left(-T_\psi^{(l)}(f_i(h_\phi(G')), f_i(h_\phi(G''))) \right) \right] \\ & - \mathbb{E}_{\mathcal{P} \times \mathcal{P}} \left[sp \left(-T_\psi^{(l)}(f_i(h_\phi(G')), f_i(h_\phi(\tilde{G}''))) \right) \right] \end{aligned} \quad (8)$$

and

$$I_{\phi, \vartheta, \psi}(H_\phi(G'), H_\phi(G''))$$

Algorithm 1 Fine-Grained Graph Contrastive Representation Learning Algorithm

Require:

Augmentations τ_α, τ_β , GNN encoders g_{ϕ_1}, g_{ϕ_2} , projection head $f_{\theta_1}, f_{\theta_2}$, batch size N , graph sets $\{G = (A, X) \mid G \in \mathcal{G}\}$.

```

1: for sampled batch  $\{G_k\}_{k=1}^N \in \mathcal{G}$  do
2:   for  $\kappa = 1$  to  $N$  do
3:      $A_k^\alpha = \tau_\alpha(A_k); A_k^\beta = \tau_\beta(A_k);$  // Augmented adjacent
       matrixes from different augmenting approaches
4:      $h_k^\alpha, H_k^\alpha = g_{\phi_1}(A_k^\alpha, X_k);$  // Node-level and graph-level
       embedding generated by GNN encoder  $\alpha$ 
5:      $h_k^\beta, H_k^\beta = g_{\phi_2}(A_k^\beta, X_k);$  // Node-level and graph-level
       embedding generated by GNN encoder  $\beta$ 
6:      $z_k^\alpha \leftarrow f_{\theta_1}(h_k^\alpha); z_k^\beta \leftarrow f_{\theta_1}(h_k^\beta);$  // Node-level repre-
       sentation mapped by the projector  $f_{\theta_1}$ 
7:      $Z_k^\alpha \leftarrow f_{\theta_2}(H_k^\alpha); Z_k^\beta \leftarrow f_{\theta_2}(H_k^\beta);$  // Graph-level
       representation mapped by the projector  $f_{\theta_2}$ 
8:   end for
9:   for  $i = 1$  to  $N$  and  $j = 1$  to  $N$  do
10:     $I_{ij}^l = T_\psi^{(l)}(z_i^\alpha, z_j^\beta); I_{ij}^g = T_\psi^{(g)}(Z_i^\alpha, Z_j^\beta);$  // Calculating
       MI of graph and nodes respectively
11:   end for
12:    $\nabla_{\phi_1, \phi_2, \theta_1, \theta_2} \frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N [\mathcal{L}(I_{ij}^l) + \mathcal{L}(I_{ij}^g)];$  // Computing
       the gradient of the total MI for backpropagation
13: end for

```

$$\triangleq \mathbb{E}_{\mathbb{P}} \left[-sp \left(-T_\psi^{(g)}(f_g(H_\phi(G')), f_g(H_\phi(G''))) \right) \right] \\ - \mathbb{E}_{\mathbb{P} \times \tilde{\mathbb{P}}} \left[sp \left(-T_\psi^{(g)}(f_g(H_\phi(G')), f_g(H_\phi(\tilde{G}''))) \right) \right] \quad (9)$$

where $\tilde{G}$ (negative graph sample) is an input sampled from a distribution $\tilde{\mathbb{P}}$, which is the same empirical probability distribution as the input probability space $\mathbb{P}$. $\tilde{G}''$ is the augmentation graph of $\tilde{G}$. Moreover, $sp(z) = \log(1 + e^z)$ is the softplus function.

Equations (8) and (9) are used to compute MI at the node and graph levels, respectively. Positive samples are drawn from the joint distribution $x_p \sim p(G', G'')$, while negative samples are drawn from the product of marginal distributions $x_q \sim p(G') \cdot p(\tilde{G}'')$. In practice, negative samples are generated by randomly shuffling the graph dataset and using other instances within each batch, following the approach in [29].

Finally, the model parameters are optimized using mini-batch stochastic gradient descent. Let $\mathcal{G}$ denote a set of training graphs, where each sample graph $G = (X, A) \in \mathcal{G}$ consists of an adjacency matrix $A \in \mathbb{R}^{|G| \times |G|}$ and initial node feature vectors $X \in \mathbb{R}^{|G| \times 13}$. The proposed contrastive learning procedure is summarized in Algorithm 1.

2) *Inference Process*: After completing the training process, we execute the lines 1 through 5 of Algorithm 1 to obtain the node-level representation $\tilde{u}_k^l = (h_k^\alpha + h_k^\beta)/2 \in \mathbb{R}^d$ and the graph-level representation $\tilde{u}_k^g = (H_k^\alpha + H_k^\beta)/2 \in \mathbb{R}^{|G_k| \times d}$ for the graph $G_k \in \mathcal{G}$, which are then used for downstream tasks.

C. Adaptive Hierarchical Clustering Classification Module

In this module, we adopt the Kitsune architecture [30] to map graph representations into combinations of several

different feature units. Thus, the fluctuation of the data is reflected by the RMSE values of multiple independent autoencoders, which are classified based on the SGM fitting. Concretely, either the flattened node-level representation $\tilde{u}^l$ or the graph-level representation $\tilde{u}^g$, both denoted as $\tilde{u} = \{u_1, u_2, \dots, u_d\} \in \mathbb{R}^d$, forms a temporal sequence $U = \{\tilde{u}_t \mid \tilde{u}_t = (u_{t,1}, u_{t,2}, \dots, u_{t,d})\}$ indexed by time t . n_t normal vectors are randomly sampled to form the training subset U_{train} .

1) *Hierarchical Clustering Map*: The combination of hierarchical clustering features filtered temporally can help to discover data patterns in the latent space. To ensure that the grouped features effectively capture traffic behavior, we follow the approach in [30], using inter-dimensional distances to quantify feature correlations. These correlation distances are incrementally aggregated and used to guide the clustering process.

More precisely, the hierarchical clustering mapping algorithm performs the following steps: The summary statistics are incrementally updated with the features of $\tilde{u}$ to generate distance matrix. Then, based on generated distance matrix, statistical hierarchical clustering is performed to form $F(\cdot)$. Finally, $F(\tilde{u}) = \vec{v}$ is computed, and $\vec{v}$ is passed to the integrated autoencoder learning module.

For the number of instances seen so far n_t , the expected value of n_t instances is $\bar{u}_{t,i} = \frac{1}{n_t} \sum_{i=0}^{n_t} u_{t,i}$, $i \in \{1, 2, \dots, d\}$.

Hence, we define the correlation distance matrix D as:

$$D_{i,j} = 1 - \frac{C_{i,j}}{\sqrt{c_i^{rs}} \sqrt{c_j^{rs}}} \quad (10)$$

where $c_i^{rs} = \sum_{t=0}^{n_t} (u_{t,i} - \bar{u}_{t,i})^2$ and $C_{i,j} = \sum_{t=0}^{n_t} ((u_{t,i} - \bar{u}_{t,i})(u_{t,j} - \bar{u}_{t,j}))$.

The next step is to perform agglomerative hierarchical clustering to find mapping function $F(\tilde{u}) = \vec{v}$ based on the distance matrix D obtained. At first, each dimension is considered as a separate cluster. Next, the most similar clusters are gradually merged according to the similarity matrix D . This process is repeated until a single cluster encompassing all d data points is formed. Agglomerative hierarchical clustering results in only one cluster, whose instructiveness for the methodology of this paper is that the combination of clusters with a maximum length $\leq s$ is intercepted and then m groups are generated in the clustering dendrogram. The feature units divided into the same groups are spliced into a vector $v_j \in \mathbb{R}^s$ and filled with zeros to the empty positions within this s -dimension space. Finally, we can obtain $\vec{v} = \{v_1, \dots, v_m\}$.

2) *Integrated Autoencoder Learning*: We use an unsupervised artificial neural network designed for anomaly detection task to depict data fluctuations [30]. It consists of two layers of autoencoders: the **Ensemble Layer** and the **Output Layer**.

In order to improve the performance and robustness of intrusion detection, each self-encoder in ensemble layer tries to reconstruct the features of the instances and forwards the reconstruction error to the self-encoder in output layer, realizing the integrated nonlinear voting mechanism and reducing the influence of noise and perturbation on the model.

Let θ denote the set of entire self-encoders. $L^{(1)} = \{\theta_1, \theta_2, \dots, \theta_m\}$ and $L^{(2)} = \theta_0$ represent the ensemble layer

and output layer, respectively, as shown in Fig. 2. Since the combination $\vec{v} = \{v_1, \dots, v_m\}$ of m features is generated from hierarchical clustering map, the ensemble layer $L^{(1)}$ is an ordered set of m three-layer autoencoders and receives feature group mapped into v_j to measure the independent anomalies by $\theta_j \in L^{(1)}$. Therein, each θ_j is a three-layer autoencoder, with input size = s . During training, $L^{(1)}$ learns the behavioral patterns of their respective subspaces by minimizing the RMSE. The output layer $L^{(2)}$ is a three-layer autoencoder with input size = m , which receives 0-1 normalized RMSE signals from $L^{(1)}$. During training, it learns the RMSE patterns of the ensemble layer, while during inference, it outputs the overall RMSE variation to support final anomaly classification.

3) *Adaptive Classification*: In [30], the output layer is responsible for generating the RMSEs, i.e. the final anomaly score. However, this approach may result in two different fluctuations canceling each other out. Therefore, we take into account the RMSE generated by the ensemble layer as well, treating all $(M + 1)$ RMSEs as a high-dimensional Gaussian distribution, and fit all RMSE to assess the degree of anomaly by using a SGM. Then, we determine the class of feature $\vec{u}$ by setting the significance level as α .

IV. EXPERIMENT

We conduct comprehensive experiments on two representative CAN datasets, as described in § IV-A. § IV-B introduces abundant comparison methods. Evaluation metrics are presented in § IV-C, followed by an analysis of hyperparameter sensitivity in § IV-D. Detection performance is comprehensively evaluated in § IV-E. § IV-F describes the vehicle-cloud collaboration framework, including two computing platforms. Finally, ablation studies are presented in § IV-G.

A. Dataset and Preprocess

Considering the representativeness and authenticity of the datasets, we choose the most widely used automotive hacking dataset in the CAN intrusion detection literature and the ROAD dataset with the highest fidelity. The two selected datasets are described in detail as follows.

1) *Car Hacking Dataset*: It contains a large number of examples of multiple fabrication attacks, generated by logging CAN traffic from OBD-II ports of real vehicles from the Hacking and Countermeasure Research Lab (HCRL) of Korea University. Specifically, the Car Hacking dataset consists of a normal dataset and four different attack datasets: DoS, Fuzzy, spoofing GEAR and spoofing RPM.

Among them, spoofing GEAR and spoofing RPM are both targeted ID attacks, targeting messages with IDs 043f and 0316, respectively. In the Car Hacking dataset, adversaries are capable of injecting arbitrary messages into the CAN bus at any time, leading to message conflicts with legitimate transmissions and periodic fluctuations in the CAN traffic. The sample distribution of the Car Hacking dataset is presented in Table I, where the relatively low proportion of labeled samples poses a significant challenge for supervised intrusion detection models, as it limits the availability of reliable training data, while also increases the difficulty of model evaluation.

TABLE II
THE DISTRIBUTION OF THE ROAD DATASET

Data Type	Normal Data Quantity	Attack Data Quantity	Attack Data Proportion	Total Data
Correlated Signal Masquerade Attack	167,974	5,453	3.14%	173,427
Max Speedometer Masquerade Attack	508,205	11,624	2.23%	519,829
Reverse Light Off Masquerade Attack	274,830	5,440	1.94%	280,270
Reverse Light On Masquerade Attack	415,239	7,930	1.87%	423,169
Max Engine Coolant Temp Masquerade Attack	57,097	43	0.075%	57,140

Since the ID and data fields in the Car Hacking dataset are stored in hexadecimal, we convert them to decimal for preprocessing. In addition, we divide all the different types of data into 70% training, 20% validation, and 10% testing sets.

2) *ROAD Dataset*: It has the advantages that real attacks are performed on real vehicles, since simulated data may not always be legitimate in real vehicles [18]. The physical effects of all captured attacks on real vehicle data are verified. The ROAD dataset contains the following five usable advanced masquerade attacks: a) Correlated Signal Masquerade Attack, b) Max Speedometer Masquerade Attack, c) Reverse Light Off Masquerade Attack, d) Reverse Light On Masquerade Attack, and e) Max Engine Coolant Temp Masquerade Attack. These attacks fall under advanced masquerade scenarios, where the attacker replaces a legitimate CAN message frame with a malicious one transmitted at the same frequency, thereby impersonating the target message. The operation in ROAD dataset is realized by extracting legitimate frames on CAN bus and injecting malicious messages in place, without changing the timing characteristics or causing message conflicts, and thus difficult to be detected by traditional methods.

Table II presents the distribution of the ROAD dataset, which contains a lower proportion of labeled samples compared to the Car Hacking dataset, thereby posing a greater challenge for intrusion detection models. On the one hand, the low proportion of attack samples undermines the model's ability to accurately distinguish malicious behaviors. On the other hand, masquerade attacks, which inject malicious content without altering the temporal patterns of message transmission, are inherently more stealthy. Together, these reasons impose more stringent requirements on model evaluation and highlight the need for label-free detection strategies.

Since the ROAD dataset contains only raw CAN logs without labels, annotations are generated using the CAN-ID algorithm [31]. To align the translated signals (CSV format) with the raw log data, we match messages based on both CAN ID and timestamp. Messages with identical ID and timestamp value are treated as a single message. The resulting data is then reformatted to match the structure of the Car Hacking dataset, followed by identical preprocessing steps.

B. Compared Methods

A fair comparison between XIPHOS and existing state-of-the-art CAN bus intrusion detection methods is challenging due to inconsistencies in detection granularity across approaches. To ensure comparability, we adopt the graph classification setting for XIPHOS, where a graph is labeled as anomalous if it contains at least one anomalous node; otherwise, it is considered as normal.

Accordingly, we compare XIPHOS with one graph-based method (G-IDCS), one classical machine learning model (RF), two basic supervised neural networks (MLP and LSTM), three advanced supervised convolutional models (EfficientNet, MobileNetV3, and CANet), and three representative unsupervised methods (DAGMM, MSFlow, and MTGFlow), covering both supervised and unsupervised paradigms.

1) *Supervised Intrusion Detection Methods:* RF [32] leverages an ensemble learning framework for multi-class classification, constructing each decision tree based on a random subset of features from individual CAN messages. Employing a feedforward neural network architecture, MLP [32] performs multi-class classification at the granularity of single CAN messages. As a prominent RNN variant, LSTM [33] utilizes the last observation window of prior CAN messages to predict the next CAN ID, enabling multi-class classification with enhanced predictive performance. EfficientNet [34] is a simple and efficient high-performance convolutional neural network after carefully balancing depth, width, and resolution. It uses an efficient composite scaling method that allows the model to easily scale the baseline ConvNet in a principled and efficient manner while satisfying resource constraints. MobileNetV3 [35] combines the depthwise separable convolutions of MobileNetV1, the inverted residual with linear bottleneck of MobileNetV2 and a lightweight attention model, and uses h-swish instead of swish as its activation function. This makes it faster in computation and more friendly to quantization. CANet [36] consists of a two-branch dense comparison module which performs multi-level feature comparison between the support image and the query image. It embeds an attention mechanism and achieves effective information fusion from multiple support examples in the setting of k-shot learning.

2) *Unsupervised Intrusion Detection Methods:* DAGMM [37] is a deep auto-encoding Gaussian Mixture Model (GMM) for unsupervised anomaly detection by combining a deep auto-encoder with GMM, which optimizes the parameters of both deep auto-encoder and the mixture model in an end-to-end fashion to help the auto-encoder get rid of less local optima. MSFlow [38] is a novel multi-scale flow-based framework composed of asymmetrical parallel flows with multi-scale perceptions, which is adopted for anomaly detection and localization according to feature distribution discrepancies. MTGFlow [39] is an unsupervised anomaly detection approach for multivariate time series anomaly detection via sample density fitting distribution, dynamic graph, and entity-aware normalizing flow, which can capture the complex distribution patterns of multivariate time series.

C. Evaluation Metrics

To evaluate the effectiveness of different methods, this paper employs classic classification metric methods. Accuracy, Precision and Recall can be calculated from the number of true negatives (TN), true positives (TP), false negatives (FN) and false positives (FP), defined as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (11)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (12)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (13)$$

In addition, the false positive rate (FPR) describes the probability of detection errors, which is the ratio of FP to the number of actual normal samples and reflects the workload of security personnel on dealing with false alarms. The true positive rate (TPR) is the ratio of TP to the number of actual attack samples.

$$\text{FPR} = \frac{FP}{FP + TN} \quad (14)$$

$$\text{TPR} = \frac{TP}{TP + FN} \quad (15)$$

The F1-score is more useful than accuracy in the case of unbalanced class distribution. It is the harmonic mean of precision and recall, calculated as follows:

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (16)$$

A common metric for dealing with unbalanced data is the Area Under the Curve (AUC) to quantitatively represent the performance of a model; it is the area under the Receiver Operating Characteristic (ROC) curve. The ROC curve is a graph with the FPR on the horizontal axis and the TPR on the vertical axis. It reveals the relationship between FPR and TPR at different thresholds. The value domain of AUC ranges from 0.5 to 1. Higher values of AUC indicate better performance of the model.

D. Hyperparameter Impact on Performance

In this subsection, we independently vary the hyperparameter variables of XIPHOS and report their impact on detection performance.

1) *Hidden Layers Size of GNN in Graph Contrastive Learning Module (h):* Node embeddings are used to encode input graph features. As can be seen in Fig. 3(a), relatively small dimensions are sufficient to encode these characteristics, while larger dimensions lead to overfitting, which may seriously affect the detection performance. Additionally, if h is too small, it will instead increase the false alarm rate. We found that $h = 32$ (Car Hacking dataset) and $h = 64$ (ROAD dataset) are the optimal dimensions.

2) *Batch Size in Graph Contrastive Learning Module (B):* The batch size B determines the amount of graph data processed during each contrast learning iteration. A small B weakens the contrastive signal and increases the risk of overfitting to a limited subset of the data. Conversely, an

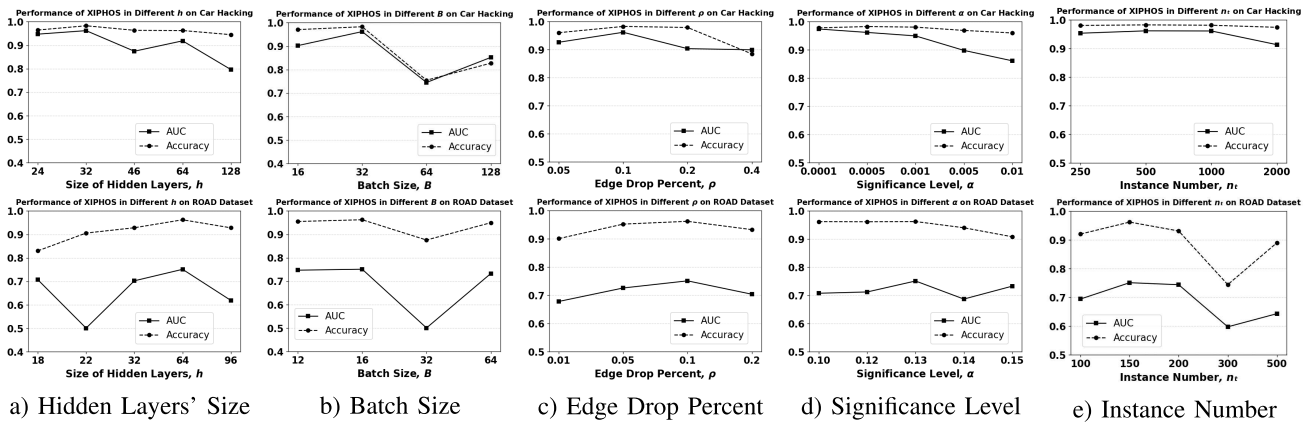


Fig. 3. Effect of different hyper-parameters on performance of XIPHOS.

excessively large B increases computational cost and may lead to overly smooth representations. As shown in Fig. 3(b), the optimal batch sizes are $B = 32$ for the Car Hacking dataset and $B = 16$ for the ROAD dataset.

3) *Edge Drop Percent of Graph Augmentation (ρ):* To minimize disturbances and improve robustness, the graph is appropriately perturbed to enhance the learning performance of the model. In Fig. 3(c), when ρ is too small, the graph-structured data features are too individualized and lack generality. Conversely, if ρ becomes too large, resulting in noise interfering with the features of normal graph data, it will also cause the detection performance to be degraded. When $\rho = 0.1$, XIPHOS can learn with the best quality augmented graph on both datasets.

4) *Significance Level of SGM (α):* The significance level, as the key to choosing the threshold, directly determines the performance of the model. Too low a significance level tends to cause false negatives, while too high a level tends to cause false positives. Fig. 3(d) shows that $\alpha = 0.0005$ (Car Hacking dataset) and $\alpha = 0.13$ (ROAD dataset) are the optimal values.

5) *Number Of Instance in Adaptive Hierarchical Clustering Classification Module (n_t):* The number of instances n_t determines the data scale used in adaptive hierarchical clustering. When n_t is too small, this module fails to learn representative features, as shown in Fig. 3(e). However, increasing n_t beyond a certain point yields diminishing returns and may lead to overfitting. To balance these effects, we empirically identify $n_t = 500$ for the Car Hacking dataset and $n_t = 150$ for the ROAD dataset as optimal settings.

E. Detection Performance

To evaluate the detection performance of XIPHOS, we conduct extensive comparative experiments on two datasets against state-of-the-art methods, without using any attack labels. The accuracy, precision, recall, F1-score, area under the ROC curve (AUC) and FPR results of all these methods including XIPHOS can be seen in Table III.

On the simple Car Hacking dataset, XIPHOS can obtain almost perfect detection results. This is because the Car Hacking dataset contains only more easily detected fabrication attacks, whose anomalous behaviour can be easily represented by hidden patterns such as periodicity. While dealing with

the ROAD dataset, XIPHOS achieves an average of 74.97% F1-score, which is significantly lower than the Car Hacking dataset. This is due to the fact that the advanced masquerade attacks in the ROAD dataset have the property of completely replacing the legitimate messages, with only the payload thresholds being altered, making them extremely stealthy. However, despite having an AUC of only 75.15%, it also outperforms the unsupervised state-of-the-art detectors and is comparable to supervised methods. The FPR of XIPHOS on two datasets is 0.19% and 0.32%, which is significantly lower than other methods possessing high F1-score.

Notably, DAGMM yields zero precision, recall, F1-score, and FPR on both datasets, with an AUC fixed at 0.5. This outcome indicates that all samples are classified as the majority class, producing no true positives and revealing the model's inability to generate discriminative anomaly scores under severe class imbalance. In contrast, XIPHOS consistently achieves stable performance across both threshold-based and ranking-based metrics, even in highly imbalanced scenarios.

F. Vehicle-Cloud Collaboration Framework

Given that the intrusion detection model serves as the core component of IVN security, it must support real-time detection and maintain a certain performance guarantee under constrained computational resources. Existing intrusion detection methods typically lack architectural designs that support adaptive deployment in heterogeneous vehicular-cloud resource environments. In practice, effective deployment requires not only a cloud-based computing platform but also an on-board detection system that ensures real-time performance, adequate computational capacity, and scalability. To meet these requirements, we select appropriate hardware based on computational capability and application scenarios, and we also describe a representative automotive-grade computing platform.

For the on-board environment, we adopt the NVIDIA Jetson Orin Nano (see Fig. 4), a pre-development Xavier system-on-chip widely used in both industry and academia for intelligent vehicle applications. It integrates a GPU and CAN bus interface, delivers approximately 21 Tera Operations Per Second (TOPS) of computing power, and offers low power consumption. Its ARM-based architecture enables the deployment and evaluation of deep learning-based intrusion

TABLE III
TEST RESULTS OF EACH METHOD IN CAR HACKING DATASET AND ROAD DATASET

Dataset	Type	Method	Accuracy	Precision	Recall	F1-score	AUC	FPR
Car Hacking Dataset	supervised	G-IDCS	0.9269	0.8759	0.8711	0.8735	0.9259	0.1450
		RF	0.9733	0.9807	0.8208	0.8303	0.9463	0.0033
		MLP	0.9314	0.9222	0.6731	0.7782	0.8317	0.0094
		LSTM	0.9415	0.7521	0.8672	0.8055	0.9512	0.0000
		EfficientNet	0.0688	0.2526	0.8015	0.1018	0.9626	0.1868
		MobileNet	0.2815	0.2474	0.8485	0.2474	0.9851	0.1865
		CANet	0.9050	0.6447	0.9725	0.7347	0.9942	0.0727
	unsupervised	DAGMM	0.9417	0.0000	0.0000	0.0000	0.5000	0.0000
		MSFlow	0.9364	0.5092	0.5621	0.5344	0.5910	0.0000
		MTGFlow	0.6848	0.5239	0.5144	0.5191	0.6398	0.0075
		XIPHOS	0.9829	0.9621	0.9645	0.9633	0.9641	0.0019
ROAD Dataset	supervised	G-IDCS	0.7281	0.3115	0.2672	0.2811	0.3276	0.1012
		RF	0.9699	0.4141	0.3429	0.6752	0.7347	0.0013
		MLP	0.8341	0.6970	0.4181	0.5227	0.7582	0.0002
		LSTM	0.6741	0.9577	0.6354	0.7639	0.7916	0.0015
		EfficientNet	0.9699	0.5281	0.5232	0.5236	0.6917	0.1358
		MobileNet	0.9637	0.5228	0.5084	0.5132	0.7615	0.0584
		CANet	0.9459	0.4111	0.3292	0.3585	0.5140	0.1667
	unsupervised	DAGMM	0.5059	0.0000	0.0000	0.0000	0.5000	0.0000
		MSFlow	0.5073	0.5596	0.5014	0.5289	0.5014	0.0500
		MTGFlow	0.5485	0.5765	0.5519	0.5639	0.6126	0.0068
		XIPHOS	0.9625	0.7478	0.7515	0.7497	0.7515	0.0032



Fig. 4. NVIDIA Jetson Orin Nano (T206).

detection algorithms under realistic, resource-constrained conditions.

To simulate a cloud environment, we utilize a high-performance computing device (LENOVO 90VA000JCP) equipped with a 64-bit Intel Core i7-13700 CPU @ 3.6GHz and an NVIDIA GeForce RTX 4080 GPU (32 GB). It not only fully trains the intrusion detection model with PyTorch 1.13 and Python 3.8, but also verifies the models' performances under non-resource-constrained conditions.

The vehicle-cloud collaborative simulation framework is designed to optimize model performance through continuous interaction between the vehicle and the cloud. All models are trained and validated in the cloud, while inference is performed on the vehicle-side platform.

The inference process consists of data loading and testing, and thus, the inference time is the sum of loading and testing time. Fig. 5 shows the inference time of various models for each CAN message. Results indicate that processing, testing and overall inference times are consistently lower on cloud platforms than on in-vehicle devices, suggesting that limited onboard resources may reduce detection speed. However, it is worth noting that the real-time inference performance

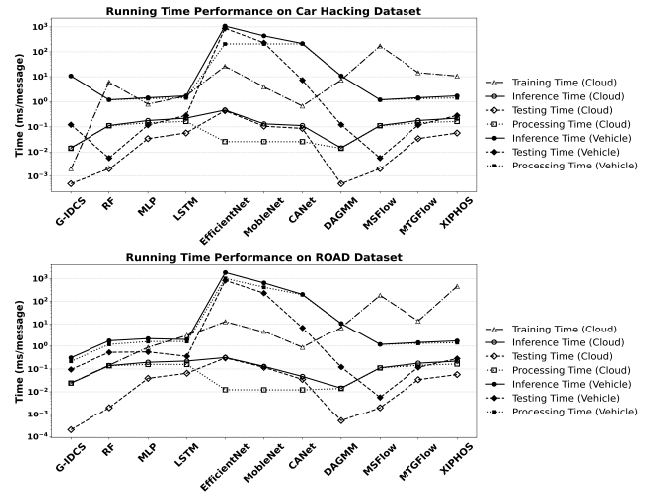


Fig. 5. Inference time of different models.

of XIPHOS on the vehicle side is still at the practically acceptable level of one millisecond. This latency is well within operational constraints for IVN intrusion detection, making the approach deployable in safety-critical environments.

G. Ablation Experiment

This section evaluates the effectiveness of key components within the XIPHOS framework by individually analyzing the impact of the Graph Contrastive Learning module (GCL) and the Adaptive Hierarchical Clustering Classification module (AHC) on overall detection performance.

1) *Upstream Feature Extractor (GCL) Replacement*: We evaluate a combination of AHC and other state-of-the-art graph representation methods including InfoGraph [27], and

TABLE IV
ABLATION EXPERIMENT RESULTS OF GCL ON CAR HACKING DATASET

Feature Extraction	<i>Downstream classifier</i>	Accuracy	Precision	Recall	F1-score	AUC	FPR
GCL	K-means	0.5031	0.6007	0.70G	0.6475	0.7021	0.4989
	FCM	0.7829	0.6390	0.7283	0.6807	0.7283	0.4365
	GMM	0.8965	0.7806	0.7618	0.7711	0.7618	0.4731
InfoGraph	AHC	0.7427	0.8639	0.5030	0.6358	0.5030	0.3246
CMVRLG		0.9157	0.9157	0.9038	0.8530	0.9038	0.1184
GCL	AHC	0.9829	0.9621	0.9645	0.9633	0.9641	0.0019

Contrastive Multi-View Representation Learning on Graphs (CMVRLG) [40]. Table IV shows the impact of these components on detection performance on the Car Hacking dataset. Notably, compared to InfoGraph and CMVRLG, XIPHOS can significantly improve the extraction of features. Furthermore, XIPHOS outperforms CMVRLG, primarily due to its enhanced ability to capture and utilize local information.

2) *Downstream Classifier (AHC) Replacement*: In this subsection, we choose to use a combination of GCL and other classical unsupervised classifiers including K-means, Fuzzy C-means (FCM), GMM. The results are shown in the first three rows of Table IV, proving the importance of the classifier for the model results. We can observe that these unsupervised classifiers are more sensitive to the initial distribution and noise of the data. Since these methods rely solely on data similarity for clustering, they face significant challenges in segmentation tasks when prior knowledge is unavailable and adaptability is limited.

The results in Table IV illustrate that the combination of GCL and AHC performs the best, which means that excellent upstream feature extraction and downstream tasks are indispensable for IVN intrusion detection.

V. DISCUSSION

A. Upstream Feature Extraction Methods

Experimental results show that transforming multidimensional temporal data into graph-structured data enables XIPHOS to capture intrinsic data characteristics more effectively. The use of MI maximization further enhances the capacity of GCL to perform deep feature extraction and structural abstraction. These results indicate that enhancing the performance of unsupervised learning hinges on constructing informative representations capable of compensating for the absence of labeled data, which remains a fundamental challenge in the field.

B. Adaptability and Quality of Training Data

AHC learns normal data behavior to detect attacks by identifying fluctuations in anomalies, as AHC can incorporate examples from different domains during training, which makes it adaptable to different environments. However, if AHC is trained on a small or low quality dataset that fails to adequately represent the behavior of IVN bus data, this may lead to an increase in false positives. Additionally, some unsupervised classification methods, such as support vector machines and

isolation forests, are more sensitive to the initial distribution and noise of the data, making it difficult to reproduce the classification results in the case of small samples. In addition, K-means, FCM and GMM may lead to unstable or unpredictable results. We leave these improvements of classification module for future research.

C. Computational Cost of Graph Transformation Steps

A quantitative analysis of the computational cost introduced by the graph transformation step is provided in Section IV. While the graph-based representation incurs moderate runtime overhead compared to lightweight baselines such as RF, LSTM, and DAGMM, it improves detection accuracy and F1-score, especially under complex and stealthy attack scenarios where traditional models often struggle to generalize. These results indicate that XIPHOS achieves a practical and deployment-oriented balance between computational efficiency and detection performance. Although not the lightest approach, its adaptability to different detection scenarios and robustness against diverse attack patterns make it well-suited in resource-constrained, safety-critical vehicular environments, where reliability and detection quality are prioritized over minimal latency. Given its substantial performance advantages, the computational overhead introduced by the graph transformation is a reasonable trade-off in practice.

D. More Advanced Attacks

The masquerade attack is executed by suspending transmissions from the targeted ECU and injecting spoofed messages from another ECU at an identical frequency. This strategy induces erroneous operations without causing message conflicts or timing anomalies, making it highly stealthy and difficult for traditional detection methods to identify. Furthermore, the scarcity of labeled samples in the ROAD dataset exacerbates the challenges of effective model training. Thus, these characteristics of advanced attacks impose stricter requirements on model evaluation and underscore the necessity of adopting unsupervised detection strategies. In Section IV, we demonstrate that XIPHOS can effectively detect masquerade attacks that imitate normal message communication patterns. However, if the adversaries have detailed knowledge of how XIPHOS works, they may craft attacks that evade detection by our model. To prevent these hostile attacks against input graphs and GNNs, developing more robust graph-based defenses against adversarial attacks remains an important direction for future research.

1) *Further Classification*: The unsupervised approach in this paper essentially performs a binary classification task that distinguishes between normal and abnormal instances. Although public IVN datasets contain a substantial amount of both labeled and unlabeled data, the available labeled data remains insufficient for training effective supervised models. Hence, self-supervised learning can further complement unsupervised methods to more precisely categorize abnormal instances into specific attack types in future work.

VI. RELATED WORK

XIPHOS focuses on three main areas of research: IVN graph-structured data conversion and utilization, unsupervised detection methods, and graph-based representation learning.

A. Graph-Structured Data Conversion

Deep learning-based IVN intrusion detection methods basically process the time-series data stream into neat Euclidean spatial data, e.g., sequences and matrices [16], [17]. However, the relationships of real-world data often do not have the canonical character of Euclidean data, which makes modeling learning with Euclidean spatial data limited.

Compared to traditional data formats, graph-structured data offers a more expressive representation for modeling complex relationships and capturing richer contextual information. It has proven effective in a variety of heterogeneous domains, such as social networks and molecular structures [41], [42], [43], due to its ability to encode dependencies through node-edge connectivity. Therefore, graph-based analytical techniques facilitate deeper exploration of the underlying patterns and structural insights within the data.

1) *Graph-Structured Data Utilization*: Meanwhile, extensive research has been conducted on graph learning tasks, including node classification, link prediction, subgraph extraction, association analysis, and graph classification [27], [29], [42], [43]. Despite its advantages in encoding rich structural contexts, the generation and analysis of graph-structured data remain challenging, particularly when compared to traditional geometric data formats. This is especially true for multi-variate time-series traffic data in networked systems, where transforming such data into semantically meaningful graph representations poses a non-trivial challenge. It is encouraging to note that several studies have begun to investigate the structural properties of CAN traffic, laying the groundwork for transforming CAN data into graph-structured representations and enabling graph-based intrusion detection, as exemplified by [22] and our prior work [25].

B. Unsupervised Detection Methods

Unsupervised intrusion detection has become a pivotal research direction for securing CAN bus systems, owing to its capacity to detect zero-day or evolving attacks without reliance on labeled training data [38], [39], [44]. Various techniques have been proposed, including autoencoder-based anomaly detection [45], temporal modeling with message attributes using LSTM [46], and statistical distribution estimation [47]. These methods reduce the dependence on costly

annotations, which is particularly beneficial in the automotive context. Nonetheless, the lack of supervision introduces several challenges. Unsupervised models are susceptible to overfitting on noisy or low-quality data, may learn unstable or non-generalizable patterns, and often depend on heuristic thresholding, thereby compromising reproducibility and hindering practical deployment.

Recent efforts have turned to graph-based approaches to enhance the structural modeling of CAN traffic. For example, the method in [44] proposes a novel graph representation to capture inter-message relationships for anomaly detection, marking a shift from traditional feature-based techniques toward structure-aware learning. However, several limitations persist. The evaluation in [44] is based on small-scale datasets focused primarily on fabrication attacks, leaving its applicability to more complex and stealthy threats such as masquerade attacks insufficiently addressed. In addition, its detection performance is relatively modest, and the experiments are conducted solely in a cloud environment, which does not account for the real-time and resource-constrained conditions typical of in-vehicle systems. Furthermore, although the approach is unsupervised, it lacks key evaluation metrics such as AUC and FPR, which are essential for assessing detection robustness and practical applicability. These shortcomings highlight the existing gap between current graph-based intrusion detection methods and their scalability in real-world automotive deployments, both in terms of implementation feasibility and comprehensive performance validation.

C. Graph-Based Representation Learning

1) *Random Walks*: A representative line of unsupervised representation learning on graph-structured data employs random-walk-based language models to capture node embeddings, as exemplified by node2vec and DeepWalk [42], [48]. These methods treat sequences generated from random walks over graphs as analogues to sentences in natural language, enabling the use of skip-gram models for embedding learning. However, to generate such sequences, random walks often simplify the graph structure, effectively trading off rich structural semantics for adjacency information [49]. Moreover, their performance is highly sensitive to hyperparameter configurations, which limits robustness and adaptability across diverse graph settings.

2) *Graph Kernels*: Another common unsupervised graph learning approach is the graph kernel [50]. Graphs are decomposed into multiple substructures and the similarity between the substructures are measured using inner products, defined as a kernel function, in appropriately normalized vector spaces. However, they require hand-drawn the way of the similarity metrics between substructures, leading to sparse or non-smooth representation and thus poor generalization performance [50].

3) *Graph Contrastive Learning*: Contrastive learning (CL) inherently aims to enforce representation consistency under appropriate transformations [51]. By minimizing the distance between augmented views of the same sample, CL has achieved notable success in self-supervised learning across

domains such as computer vision and natural language processing [52], [53], and has attracted growing research interest in recent years. In graph learning, where data often lacks sufficient labels or is difficult to annotate, CL has been widely adopted to learn meaningful node and graph representations in an unsupervised manner. This series of algorithm studies is known as Graph Contrastive Learning (GraphCL) methods.

GraphCL has emerged as a leading paradigm for unsupervised representation learning at both the node and graph levels [26], [27], [29], [40], [43]. The foundational work on MI estimation, introduced in [28] and [54], provided the theoretical basis for subsequent methods such as [27] and [29]. These approaches optimize GNN encoders by maximizing the MI between augmented views of the same graph, thereby encouraging semantic consistency, while simultaneously pushing apart representations of different graphs. The encoder parameters are updated iteratively until convergence or a predefined termination criterion is satisfied [29].

Nevertheless, several prior works [27], [29], [43] do not incorporate explicit graph augmentations. Such augmentations serve as a form of structured noise injection, which can enhance model robustness and preserve intrinsic graph properties. While the work in [43] inspires us to use graph contrastive learning paradigm as an upstream unsupervised feature extraction strategy, the research [40] further presents a graph diffusion-based augmentation method. However, it lacks a comprehensive formulation of local-global MI objectives and its node representations are insufficiently expressive, with experimental validation limited to graph-level tasks but lacking node classification capabilities.

VII. CONCLUSION

We propose an unsupervised and adaptive intrusion detection mechanism, XIPHOS, capable of operating without attack labels. With no dependence on a priori knowledge of attack patterns, XIPHOS dynamically adapts to unknown threats, making it particularly suitable for real-world in-vehicle environments. A novel MI computation strategy is introduced, which integrates local and global information to enhance feature extraction. By constructing graph-structured representations and maximizing MI, XIPHOS captures abstract features and performs intrusion detection through an adaptive hierarchical classification framework. Vehicle-cloud collaborative experiments on two widely used CAN datasets demonstrate the effectiveness of the proposed method in achieving satisfactory results in terms of detection performance.

REFERENCES

- [1] H. H. Jeong, Y. C. Shen, J. P. Jeong, and T. T. Oh, "A comprehensive survey on vehicular networking for safe and efficient driving in smart transportation: A focus on systems, protocols, and applications," *Veh. Commun.*, vol. 31, no. 100349, pp. 1–22, Oct. 2021.
- [2] B. Lampe and W. Meng, "Intrusion detection in the automotive domain: A comprehensive review," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 4, pp. 2356–2426, 2023.
- [3] Y. Xie, G. Zeng, R. Kurachi, F. Xiao, H. Takada, and S. Hu, "Timing analysis of CAN FD for security-aware automotive cyber-physical systems," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 3064–3078, Jul. 2023.
- [4] Bosch, Robert Bosch GmbH, Postfach, Gerlingen, Germany, 1991.
- [5] L. Xue et al., "Said: State-aware defense against injection attacks on in-vehicle network," in *Proc. 31st USENIX Secur. Symp.*, Boston, MA, USA, Aug. 2022, pp. 1921–1938.
- [6] Y. Liu et al., "Vehicular intrusion detection system for controller area network: A comprehensive survey and evaluation," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 7, pp. 10979–11009, Jul. 2025.
- [7] L. Wang, Q. Zhao, W.-B. Lee, and C. Wang, "Deploying intrusion detection on in-vehicle networks: Challenges and opportunities," *IEEE Netw.*, vol. 39, no. 1, pp. 306–312, Jan. 2025.
- [8] J. Cao et al., "Anomaly detection for in-vehicle network using self-supervised learning with vehicle-cloud collaboration update," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7454–7466, Jul. 2024.
- [9] S. Jeong, M. Ryu, H. Kang, and H. K. Kim, "Infotainment system matters: Understanding the impact and implications of in-vehicle infotainment system hacking with automotive grade Linux," in *Proc. 13th ACM Conf. Data Appl. Secur. Privacy*, Apr. 2023, pp. 201–212.
- [10] Z. Lu, Q. Wang, X. Chen, G. Qu, Y. Lyu, and Z. Liu, "LEAP: A lightweight encryption and authentication protocol for in-vehicle communications," in *Proc. IEEE Intell. Transp. Syst. Conf. (ITSC)*, Auckland, New Zealand, Oct. 2019, pp. 1158–1164.
- [11] S. Rajapaksha, H. Kalutarage, M. O. Al-Kadri, A. Petrovski, G. Madzudzo, and M. Cheah, "AI-based intrusion detection systems for in-vehicle networks: A survey," *ACM Comput. Surveys*, vol. 55, no. 11, pp. 1–40, Feb. 2023.
- [12] F. Luo, Z. Yang, Z. Zhang, Z. Wang, B. Wang, and M. Wu, "A multi-layer intrusion detection system for SOME/IP-based in-vehicle network," *Sensors*, vol. 23, no. 9, p. 4376, Apr. 2023.
- [13] H. Sun, J. Wang, J. Weng, and W. Tan, "KG-ID: Knowledge graph-based intrusion detection on in-vehicle network," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 4, pp. 4988–5000, Apr. 2025.
- [14] H. Alqahtani and G. Kumar, "Deep learning-based intrusion detection system for in-vehicle networks with knowledge graph and statistical methods," *Int. J. Mach. Learn. Cybern.*, vol. 16, nos. 5–6, pp. 3539–3555, Jun. 2025.
- [15] X. Zhou, W. Liang, J. She, Z. Yan, and K. I. Wang, "Two-layer federated learning with heterogeneous model aggregation for 6G supported Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 5308–5317, Jun. 2021.
- [16] K. Wang, A. Zhang, H. Sun, and B. Wang, "Analysis of recent deep-learning-based intrusion detection methods for in-vehicle network," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 1843–1854, Feb. 2023.
- [17] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, pp. 1–13, Jan. 2020.
- [18] M. E. Verma et al., "A comprehensive guide to CAN IDS data and introduction of the ROAD dataset," *PLoS ONE*, vol. 19, no. 1, Jan. 2024, Art. no. e0296879.
- [19] M. H. Shahriar, W. Lou, and Y. T. Hou, "CANtropy: Time series feature extraction-based intrusion detection systems for controller area networks," in *Proc. Inaugural Int. Symp. Vehicle Secur. Privacy*, 2023, pp. 1–8.
- [20] K.-T. Cho and K. G. Shin, "Error handling of in-vehicle networks makes them vulnerable," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA: Association for Computing Machinery, Oct. 2016, pp. 1044–1055.
- [21] C. Miller and C. Valasek, "Advanced can injection techniques for vehicle networks," in *Proc. Black Hat USA*, Las Vegas, NV, USA, Aug. 2016, pp. 1–28.
- [22] R. Islam, R. U. D. Refat, S. M. Yerram, and H. Malik, "Graph-based intrusion detection system for controller area networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 3, pp. 1727–1736, Mar. 2022.
- [23] J. B. Kinney and G. S. Atwal, "Equitability, mutual information, and the maximal information coefficient," *Proc. Nat. Acad. Sci. USA*, vol. 111, no. 9, pp. 3354–3359, Mar. 2014.
- [24] S. Nowozin, B. Cseke, and R. Tomioka, "f-gan: Training generative neural samplers using variational divergence minimization," in *Proc. 30th Conf. Neural Inf. Process. Syst.*, Barcelona, Spain, Dec. 2016, pp. 271–279.
- [25] K. Wang, Q. Jiang, B. Wang, Y. Wu, and H. Zhang, "STATGRAPH: Effective in-vehicle intrusion detection via multi-view statistical graph learning," 2023, *arXiv:2311.07056*.
- [26] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS)*, Vancouver, BC, Canada, Dec. 2020, p. 5812.
- [27] F.-Y. Sun, J. Hoffmann, V. Verma, and J. Tang, "InfoGraph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization," in *Proc. 8th Int. Conf. Learn. Represent.*, Addis Ababa, Ethiopia, Apr. 2019, pp. 1–14.

- [28] R. D. Hjelm et al., "Learning deep representations by mutual information estimation and maximization," in *Proc. Int. Conf. Learn. Represent.*, New Orleans, LA, USA, May 2019, pp. 1–24.
- [29] P. Velickovic, W. Fedus, W. L. Hamilton, P. Lio, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–17.
- [30] Y. Mirsky, T. Doitsman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, Feb. 2018, pp. 1–15.
- [31] M. E. Verma, R. A. Bridges, J. J. Sosnowski, S. C. Hollifield, and M. D. Iannacone, "CAN-D: A modular four-step pipeline for comprehensively decoding controller area network data," *IEEE Trans. Veh. Technol.*, vol. 70, no. 10, pp. 9685–9700, Oct. 2021.
- [32] T. Moulahi, S. Zidi, A. Alabdulatif, and M. Atiquzzaman, "Comparative performance evaluation of intrusion detection based on machine learning in in-vehicle controller area network bus," *IEEE Access*, vol. 9, pp. 99595–99605, 2021.
- [33] I. Berger, R. Rieke, M. Kolomeets, A. Chechulin, and I. Kutenko, "Comparative study of machine learning methods for in-vehicle intrusion detection," in *Proc. Int. Workshop Secur. Privacy Requirements Eng.*, 2019, pp. 85–101.
- [34] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 1–10.
- [35] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1314–1324.
- [36] C. Zhang, G. Lin, F. Liu, R. Yao, and C. Shen, "CANet: Class-agnostic segmentation networks with iterative refinement and attentive few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 5217–5226.
- [37] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Represent.*, Feb. 2018, pp. 1–19.
- [38] Y. Zhou, X. Xu, J. Song, F. Shen, and H. T. Shen, "Msflow: Multi-scale flow-based framework for unsupervised anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 2, pp. 2437–2450, Feb. 2024, doi: [10.1109/TNNLS.2023.3344118](https://doi.org/10.1109/TNNLS.2023.3344118).
- [39] Q. Zhou, S. He, H. Liu, J. Chen, and W. Meng, "Label-free multivariate time series anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3166–3179, Jul. 2024.
- [40] K. Hassani and A. H. Khasahmadi, "Contrastive multi-view representation learning on graphs," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 4116–4126.
- [41] W. Jin, R. Barzilay, and T. S. Jaakkola, "Junction tree variational autoencoder for molecular graph generation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, J. G. Dy and A. Krause, Eds., 2018, pp. 2328–2337.
- [42] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, San Francisco, CA, USA, Aug. 2016, pp. 855–864.
- [43] J. Qiu et al., "GCC: Graph contrastive coding for graph neural network pre-training," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 1150–1160.
- [44] N. Kabilan, V. Ravi, and V. Sowmya, "Unsupervised intrusion detection system for in-vehicle communication networks," *J. Saf. Sci. Resilience*, vol. 5, no. 2, pp. 119–129, Jun. 2024.
- [45] S. Longari, D. H. Nova Valcarcel, M. Zago, M. Carminati, and S. Zanero, "CANnol: An anomaly detection system based on LSTM autoencoders for controller area network," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1913–1924, Jun. 2021.
- [46] V. Tanksale, "Design of anomaly detection functions for controller area networks," *IEEE Open J. Intell. Transp. Syst.*, vol. 2, pp. 312–321, 2021.
- [47] H. Narasimhan, V. Ravi, and N. Mohammad, "Unsupervised deep learning approach for in-vehicle intrusion detection system," *IEEE Consum. Electron. Mag.*, vol. 12, no. 1, pp. 103–108, Jan. 2023.
- [48] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2014, pp. 701–710.
- [49] L. F. R. Ribeiro, P. H. P. Saverese, and D. R. Figueiredo, "struc2vec: Learning node representations from structural identity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Halifax, NS, Canada, Aug. 2017, pp. 385–394.
- [50] A. Narayanan, M. Chandramohan, R. Venkatesan, L. Chen, Y. Liu, and S. Jaiswal, "Graph2vec: Learning distributed representations of graphs," 2017, *arXiv:1707.05005*.
- [51] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
- [52] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9729–9738.
- [53] T. Mikolov, L. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 2, Dec. 2013, pp. 3111–3119.
- [54] M. I. Belghazi et al., "Mutual information neural estimation," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 531–540.

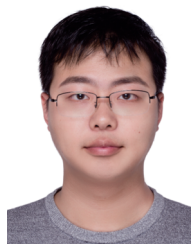


Qiguang Jiang received the master's degree in computer science and technology with Harbin Institute of Technology (HIT), China. Her research interests include intelligent and efficient in-vehicle intrusion detection models.



Kai Wang (Member, IEEE) received the Ph.D. degree in communication and information systems from Beijing Jiaotong University, China, in 2014. He is currently a Full Professor with the School of Computer Science and Technology, Harbin Institute of Technology, Weihai, China, and also with the Faculty of Computing, Harbin Institute of Technology, Harbin, China. He has published more than 40 articles in prestigious international journals, including IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, ACM TOIT, and ACM TIST.

His research interests include in-vehicle network security, advanced persistent threat (APT) detection, and trustworthy machine learning. He is a member of ACM and a Senior Member of China Computer Federation (CCF).



Yuliang Wei received the Ph.D. degree from the School of Computer Science and Technology, Harbin Institute of Technology, China, in 2006. He is currently an Associate Professor with the School of Computer Science and Technology, Harbin Institute of Technology, Weihai, China. He has published more than 20 papers in prestigious international journals and conferences. His research interests include machine learning and data analysis.



Hongri Liu received the Ph.D. degree from the Faculty of Computing, Harbin Institute of Technology (HIT), China, in 2021. He is currently an Assistant Researcher with the School of Computer Science and Technology, HIT, Weihai, China. His research interests include industrial control network security, V2X security, and zero trust security.



Bailing Wang (Member, IEEE) received the Ph.D. degree in computer architecture from Harbin Institute of Technology (HIT), China, in 2006. He is currently a Full Professor with HIT, Weihai, and Qingdao Research Institute, Qingdao, China. He has published more than 80 articles in prestigious international journals and been selected for China National Talent Plan. His research interests include information content security and industrial control network security.